COMP 551: Logistic Regression Project Report

Aymane Radouane (261231812) & Nickolai Tsoukanov (261161485)
& Williams Lendjoungou (261167713) & Team Members

February 28, 2026

Abstract

This report investigates logistical regression using stochastic gradient descent optimization with mini-batch implementation on a 57 continuous features, attempting to predict whether email are spam or not. Through a systematic grid search and 5-fold cross-validation, we identify an optimal hyperparameter configuration ($\eta = 0.1$, batch size 16) that achieves a mean accuracy of 0.9382. Our analysis explores the bias-variance tradeoff through an L2 regularization λ-sweep and finds feature selection via L1 regularization paths. An optimal value for λ=0.001. Results demonstrate that even though smaller batch sizes are computational efficient they introduce significant instability. Also noting L2 regularization effectively stabilizes training dynamics and prevents overfitting. Furthermore, L1 analysis demonstrates a clear trade-off between model sparsity and predictive performance, with peak accuracy occurring as regularization allows more features to enter the model.

1 Introduction

This project addresses the issue of spam emails infiltrating email inboxing. A dataset with 57 features was given to train a model using logistic regression. The dataset itself hade 48 word frequency counts, 6 character frequency counts and 3 capital run length attributes. The logistic model will using stochastic gradient descent and a mini-batch approach to fit to the dataset. Cross Validation will be done for hyper parameter tuning, and a λ-sweep to optimization regularization. L1 and L2 regularization will be investigated.

2 Data Preprocessing

- **Validating Types** All types in the dataset were floating point or integer values. No null values were found anywhere in the table.
- **IQR Threshold** Outliers values were clipped using the IQR test with a 1.5 multiplier threshold. When the IQR was already 0, nothing was clipped.
- **Standardization (Z-Normalization)** Standardization had to be applied to all 57 features to have them on a comparable scales and for numerical stability. A common issue of dividing by a zero standardization came up often when managing word frequency. In those case the standard deviation was set to one.
- **One Hot Encoding** No one hot encoding was necessary for this dataset, all features were continuous.

- **Train/Test Split** As by instructions, "np.random.default rng(RANDOM SEED=2026)" was used for seperations to guarantee reproducible results.

3 Methods

3.1 Logistic Regression with SGD and Mini-Batch Implementation

The entire core and machinery of the model sits inside one python class: "LogisticRegression". This class gives access to all modifiable options necessary for hyper-parameter training such as: regularization type, regularization constant, learning rate, max epochs, batch size, epsilon (optimization threshold), a record weight history option, and bias option (include or don't include bias in regularization).

As convention, bias was excluded from regularization. It was of the form:

$$J_{reg} = \frac{\lambda}{2}\|w\|_2^2$$

Weighs were stored per iteration rather than per epoch. Equation 1 was used to update said weights at every iteration. Where η is the learning rate, m is the batch size, and $\mathbf{w}'$ denotes the weight vector with the bias term w_0 excluded from the regularization penalty ($\mathbf{w}' = [0, w_1, w_2, \ldots, w_n]$). λ is the regularization constant, and $p(\mathbf{w}', \lambda)$ is the gradient of the regularization function.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\left(\frac{1}{m}\sum_{i=1}^{m}(\sigma(\mathbf{x}_i \cdot \mathbf{w}) - y_i)\mathbf{x}_i + p(\mathbf{w}', \lambda)\right) \tag{1}$$

3.2 Cross-Validation & Hyperparameter Tuning

Model selection was performed using 5-fold cross-validation on the training split. For each configuration, mean cross-entropy and mean accuracy were averaged across folds. The test set was not used during tuning. The parameter space overwhich hyperparameter tuning was done can be seen below in table 1. For each of those 18c configurations, 5-fold CV's were performed. Training 90 models in total to find the best combination.

Hyperparameter	**Search Space Values**
Learning Rate (η)	$\{0.1, 0.01, 0.001\}$
Batch Size	$\{16, 64\}$
Epochs	$\{50, 80, 110\}$

Table 1: Hyperparameter Grid Search Space

3.3 L1-Regularized Logistic Regression

To analyze feature selection behavior, we additionally trained an L1-regularized logistic regression model using `scikit-learn`'s `LogisticRegression` implementation with the `saga` solver. A logarithmically spaced grid of regularization parameters $C \in [10^{-4}, 10^4]$ was evaluated, where $C = 1/\lambda$. For each C, the model was fit on the training split, the coefficient vector was recorded, and the number of non-zero coefficients ($\|w\|_0$) was computed to measure sparsity.

Five-fold cross-validation was used to select the optimal C based on mean validation accuracy. The final model was then refit on the full training set and evaluated on the held-out test set.

4 Results

4.1 Training Curves

For visualization, 6 models are were trained models with the optimally found learning rate of $\eta = 0.1$ and batch sizes = 1, 16, 64 for both L2 regularization and without. The λ value used for L2 regularization was 0.1 to exaggerate its affects during on the model. In the figure below, the cross-entropy was calculated as an average per epoch.

(a) Batch size = 1 (b) Batch size = 16 (c) Batch Size = 64

Figure 1: Comparison of Logistic Regression training dynamics across various batch sizes and regularization settings.

4.2 Hyperparameter Tuning

The best hyperparameter configuration can be found in table 2. The learning rate had the strongest effect on performance. A higher learning rate (0.1) yielded faster convergence and lower cross-entropy, while smaller learning rates resulted in slightly worse performance. Batch size had a moderate impact, with batch size 16 consistently outperforming 64. Increasing epochs beyond 80 produced minimal gains.

LR (η)	Batch Size	Epochs	Mean CE	Mean Accuracy
0.1	16	80	0.1907	0.9382

Table 2: Optimal Hyperparameters from 5-Fold Cross-Validation

4.3 Bias-Variance via λ sweep.

Table 3: Fixed Hyperparameters for λ Sweep

Hyperparameter	Value
Learning Rate (η)	0.1
Batch Size	16
Max Epochs	200

As shown in Figure 2, the λ-sweep clearly illustrates the bias–variance tradeoff. With $\lambda = 0$, the model achieves low training cross-entropy but exhibits a small train–validation gap, indicating limited variance. The lowest validation cross-entropy occurs at $\lambda = 0.001$, providing the best generalization by slightly reducing variance without significantly increasing bias. As λ increases

Table 4: Fixed Hyperparameters for λ Sweep

Hyperparameter	**Value**
Learning Rate (η)	0.1
Batch Size	16
Max Epochs	200

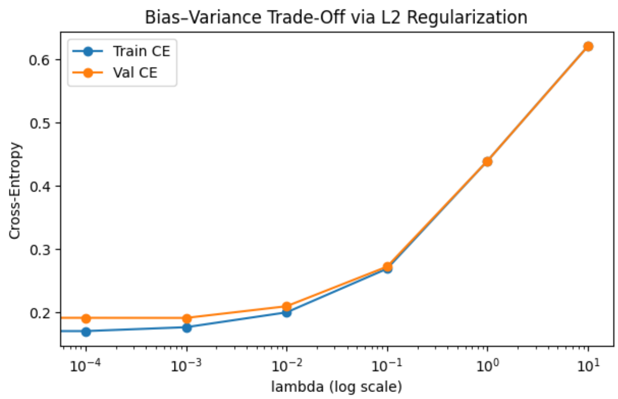


Figure 2: Figure 2: Bias–variance trade-off under L2 regularization. Training and validation cross-entropy are plotted against λ (log scale).

beyond 10^{-2}, both training and validation losses rise steadily, demonstrating underfitting due to excessive regularization (high bias). Overall, the curve follows the expected U-shaped behavior, with small regularization yielding optimal performance and large λ degrading model capacity.

4.4 L1 Regularization Path and Sparsity Analysis

To study feature selection effects, we trained an L1-regularized logistic regression model across a grid of C values (where $C = 1/\lambda$). The regularization path illustrates how coefficients evolve as regularization weakens.

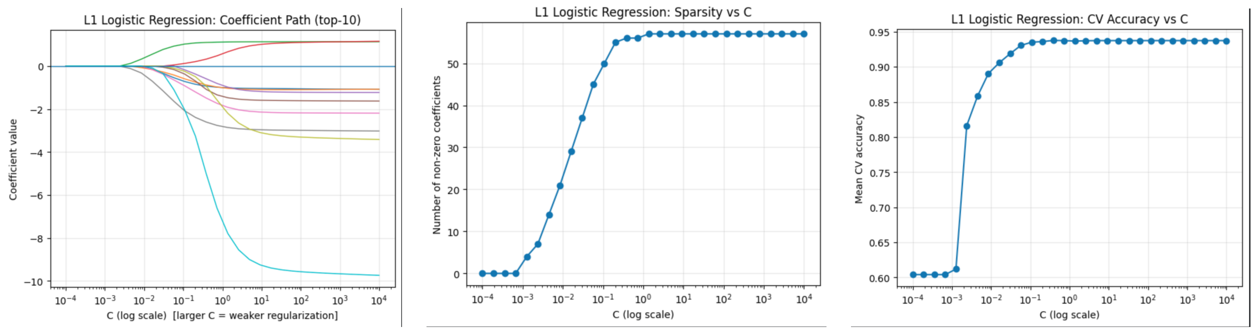
At very small C (strong regularization), all coefficients are driven to zero, resulting in a highly sparse model. As C increases, coefficients gradually become non-zero, indicating that features enter the model in order of predictive importance.

The sparsity plot confirms that the number of active features increases monotonically with C. Under strong regularization, the model selects very few features, while weak regularization results in nearly all features being retained.

Cross-validation accuracy increases sharply as regularization weakens and plateaus for moderate C values. The best C obtained via cross-validation achieved a test accuracy of approximately 0.90. Notably, maximum performance was obtained when most features were included, illustrating the trade-off between sparsity (interpretability) and predictive performance.

5 Discussion and Conclusion

In the training curves figure 1, there is clear evidence that a batch size of 1, although highly more computationally efficient, may produce highly unstable results. This is seen with the none regularized model diverging in cross-entropy, increasing as the iterations increased. As batch size increase, we see higher stability, with a slow drop off with at a batch size of 64. L2 Regularization,



(a) Coefficient Path (b) Sparsity vs C (c) CV Accuracy vs C

Figure 3: L1 regularization analysis: coefficient path, sparsity, and cross-validation accuracy.

the orange line, show a higher stability, although having a higher cross-entropy error. Signifying a restraint against overfitting, exactly what it is used for.

5.1 Regularization

The results confirm the expected role of L2 regularization in controlling generalization. For small λ, training and validation cross-entropy remain low with a minimal gap, indicating low variance and strong generalization. As λ increases, both losses rise steadily, reflecting increasing bias and eventual underfitting. The optimal performance occurs in the small-λ regime, where regularization slightly improves stability without overly restricting model capacity.

L1 vs L2 Regularization. Compared to L2 regularization, L1 induces sparsity by driving some coefficients exactly to zero. While L2 shrinks coefficients smoothly, L1 performs implicit feature selection. In our experiments, L1 produced highly sparse models under strong regularization, but optimal predictive performance was achieved when most features were retained. This highlights the classical trade-off between interpretability and predictive accuracy.

Statement of Contributions

Nickolai worked on Part 1, Williams worked on parts 2 and 3, and Aymane completed part 4. All authors worked collaboratively on the report, the code and reviewed all pieces.

Statement on the use of LLMs

LLM use was very limited during this project.